Cafeteria Billing and Inventory System

Group 8

Qamar Abbas Musvi CS24B1068
Pruthvi Ram CS24B1022
Valmiki Vinay CS24B1040

Project Overview

A C++ billing system designed for cafeteria operations, implementing core Object-Oriented Programming concepts including classes, inheritance, polymorphism, operator overloading, template programming, and dynamic memory management.

Table of Contents

1. Features
2. Class Descriptions
3. OOP Concepts Demonstrated
4. File Structure
5. Compilation
6. Usage Guide
7. Code Explanation
8. Key Design features
9. Contributions
10. Sample Output

Features

- Menu Management: Display available items with prices and stock.
- Order Processing: Add multiple items to orders with quantity validation
- Payment Processing: Support for multiple payment methods (Cash and UPI)
- Inventory Management: Real-time stock tracking and updates using template classes
- Item Management: Add new items,Delete existing items and restock existing inventory
- Daily Summary: Track total orders and earnings using static members
- Stock Validation: Prevents orders exceeding available quantity

- Error Handling: Input validation for payment methods and item quantities

---

System Architecture

Core Classes

1. Item Class (`item.h`, `item.cpp`)

Represents individual menu items with:

- Attributes:
    - itemID: Unique identifier for each item
    - name: Item name
    - price: Item price
    - quantity: Available stock
- Methods:
    - getID(), getName(), getPrice(), getQuantity(): Accessor methods
    - addStock(int added): Increases stock quantity
    - updateStock(int sold): Reduces stock after sale
- Operator Overloading:
    - Overloaded << operator for formatted output display

```
class Item
{
    int itemID;//unique id for each item
    string name;
    double price;
    int quantity;//Available Quantity
public:
    Item(int id = 0, string n = "", double p = 0, int q = 0);
    int getID() const;
    string getName() const;
    double getPrice() const;
    int getQuantity() const;
    void updateStock(int sold);//update quanity after item is sold
    void addStock(int added);//update quanity when the item is restocked
    friend ostream& operator<<(ostream& out, const Item& i);
};
#endif
```

---

2. Inventory Template Class (`inventory.h`, `inventory.cpp`)

Generic template class for managing collections of items:

- Template Declaration: template <typename T>
- Storage: Vector-based storage (vector<T> stock)
- Methods:
    - showMenu(): Displays all items in inventory
    - getItemByName(string name): Returns pointer to item by name
    - getItemByID(int id): Returns pointer to item by ID
    - updateItemStock(int id, int qty): Updates stock after sale
    - restockItem(int id, int qty): Adds stock to existing items
    - deleteItem(int id): Removes an item from the stock vector.
- Operator Overloading:
    - Overloaded += operator to add items: inv += Item(...)

```
template <typename T>
class Inventory
{
    vector<T> stock;
public:
    void showMenu();
    T* getItemByName(string name);
    T* getItemByID(int id);
    void updateItemStock(int id, int qty);
    void restockItem(int id, int qty);
    void deleteItem(int id);
    Inventory<T>& operator+=(const T& i);
};
#endif
```

---

3. Payment Class Hierarchy (`payment.h`, `payment.cpp`)

Abstract base class demonstrating inheritance and polymorphism:
Base Class: Payment

- Abstract: Contains pure virtual function makePayment()
- Protected member: float amount
- Virtual destructor: Ensures proper cleanup in derived classes

Derived Classes:

1. CashPayment:
    - Validates cash amount entered
    - Calculates and returns change
    - Handles insufficient payment scenarios
    - Input validation with exception handling
2. UpiPayment:
    - Validates UPI ID format (minimum 5 characters)
    - Simulates UPI payment processing
    - Provides user feedback

```
class Payment
{
protected:
    float amount;// Amount to be paid
public:
    Payment(float amt=0);
    virtual bool makePayment() = 0; // Pure virtual
    virtual ~Payment(); // Virtual destructor
};
class CashPayment : public Payment
{
public:
    CashPayment(float amt=0);
    bool makePayment() override; // Handle cash payment
};
class UpiPayment : public Payment
{
public:
    UpiPayment(float amt=0);
    bool makePayment() override;// Handle UPI payment
};
#endif
```

---

4. Order Class (`order.h`, `order.cpp`)

Handles customer orders and payment processing:

- Static Members:
    - static int totalOrders: Tracks total orders
    - static float totalEarnings: Tracks total revenue
- Instance Members:
    - orderNo: Unique order identifier
    - vector<Item> orderItems: Items in the order
    - vector<int> qty: Quantities for each item
    - float totalAmount: Total bill amount
    - Payment *paymentMethod: Polymorphic payment handler
- Methods:
    - addItem(const Item& i, int q): Adds item to order
    - calculateTotal(): Computes total amount
    - printBill(Inventory<Item>& inv): Displays bill and processes payment
    - static showDailySummary(): Shows daily statistics

```
class Order// Handles customer orders
{
    static int totalOrders;   // Total orders placed today
    static float totalEarnings; // Total Earning
    int orderNo;                // Order ID
    vector<Item> orderItems;    // Items in current order
    vector<int> qty;            // Quantity for each item
    float totalAmount;          // Final bill amount
    Payment *paymentMethod;     // Payment handler (Cash/UPI)

public:
    Order();
    void addItem(const Item& i, int q); // Add item to order
    void calculateTotal(); // Compute totalAmount
    void printBill(Inventory<Item> &inv);// takes the specific Inventory<Item>
    static void showDailySummary(); // Day-end summary
    ~Order(); // Cleanup paymentMethod
};

#endif
```

---

5. BillingSystem Class (`bill.h`, `bill.cpp`)

Main controller coordinating all operations:

- Composition: Contains `Inventory<Item> inv` object
- Constructor: Initializes inventory with default items
- Methods:
    - start(): Main menu loop
    - takeOrder(): Processes customer orders
    - addNewItem(): Adds new items to inventory
    - restockExistingItem(): Restocks items
    - deleteExistingItem(): Handles the user interface for deleting an item,

Menu Options:

1. Show Menu
2. Take Order
3. Daily Summary
4. Add New Item
5. Restock Item
6. Delete Item
7. Exit

```
#ifndef BILL_H
#define BILL_H
#include "inventory.h"
#include "item.h"
#include <string>
using namespace std;
class BillingSystem
{
    Inventory<Item> inv;
public:
    void takeOrder();
    void addNewItem();
    void restockExistingItem();
    BillingSystem();
    void deleteExistingItem();
    void start();
};
#endif
```

Object-Oriented Programming Concepts Demonstrated

1. Encapsulation

- All classes have private data members
- Public methods provide controlled access
- Example: Item class hides itemID, price, quantity behind getters

2. Abstraction

- Payment class provides abstract interface for payment processing
- BillingSystem hides complex logic behind simple menu interface
- Users interact with high-level operations without knowing implementation details

3. Inheritance

- CashPayment and UpiPayment inherit from Payment base class
- Public inheritance demonstrates "is-a" relationship
- Derived classes override pure virtual makePayment() method

4. Polymorphism

- Runtime polymorphism through virtual functions
- Base class pointer (Payment *paymentMethod) calls correct derived class method

5. Operator Overloading

- Inventory::operator+=: Adds items with inv += item
- operator<< for Item: Enables cout << item

6. Templates

- Inventory<T> is a generic template class
- Can work with any type (currently instantiated for Item)
- Requires explicit instantiation: template class Inventory<Item>;

7. Composition

- BillingSystem contains Inventory<Item> object
- Order contains vectors of Item objects

8. Dynamic Memory Management

- new and delete for Payment objects in Order class
- Proper cleanup in destructor prevents memory leaks

9. Static Members

- Order::totalOrders and Order::totalEarnings shared across all instances

- Static method showDailySummary() accesses static data
- Enables tracking global statistics

10. Friend Functions

- operator<< declared as friend in Item class
- Allows access to private members for formatted output

11. Exception Handling

- Inventory throws runtime_error for invalid item deletions, ensuring data integrity during administrative tasks.
- Payment uses try-catch blocks to trap conversion errors from stof, preventing crashes from non-numeric user input..
- BillingSystem catches all exceptions to display user-friendly messages, maintaining a robust and crash-resistant application flow.

File Structure

Header Files (.h)

1. item.h - Item class declaration
2. inventory.h - Inventory template class declaration
3. payment.h - Payment class hierarchy declarations
4. order.h - Order class declaration
5. bill.h - BillingSystem class declaration

Implementation Files (.cpp)

1. item.cpp - Item class implementation
2. inventory.cpp - Inventory template class implementation
3. payment.cpp - Payment classes implementation
4. order.cpp - Order class implementation
5. bill.cpp - BillingSystem class implementation
6. main.cpp - Program entry point

Compilation

Compilation Command:

g++ main.cpp item.cpp inventory.cpp order.cpp payment.cpp bill.cpp

---

Default Menu Items

The system initializes with the following items:

Item ID	Name	Price (Rs.)	Initial Stock
101	Burger	80	20
102	Pizza	150	10
103	Fries	60	15
104	Coke	40	30

Usage Guide

Taking an Order:

1. Select option 2 (Take Order)
2. View the displayed menu
3. Enter Item ID from the menu
4. Enter desired quantity

5. Repeat for multiple items
6. Enter 0 to finish adding items
7. Choose payment method:
    - 1 for Cash (enter amount, receive change)
    - 2 for UPI (enter UPI ID)
8. Complete payment to confirm order

Adding New Items:

1. Select option 4 (Add New Item)
2. Enter unique Item ID
3. Provide item name
4. Enter price
5. Specify initial stock quantity

Deleting Existing Items:

1. Select option 6(Delete Item)
2. Enter ID of item to be deleted

Restocking Items:

1. Select option 5 (Restock Item)
2. View current menu
3. Enter Item ID to restock
4. Specify quantity to add

Viewing Daily Summary:

- Select option 3 to view:
    - Total number of orders
    - Total earnings for the day

Key Design Features

Error Handling

- Stock Validation: Prevents orders exceeding available quantity
- Input Validation: Validates payment inputs (cash amount, UPI ID)
- Duplicate ID Check: System checks for unique item IDs

Real-time Updates

- Inventory automatically updates after successful orders
- Stock quantities decrease after sales
- Failed payments don't affect inventory

Payment Validation

- Cash: Verifies sufficient amount before processing
- UPI: Validates UPI ID format (minimum 5 characters)
- Retry Logic: Allows users to correct invalid inputs

Extensibility

- Easy to add new payment methods by inheriting from `Payment` class
- Template-based inventory can be adapted for different item types

---

Code Explanation

Order Processing Flow:

1. User selects "Take Order"
    - BillingSystem::takeOrder() called
2. Menu Display
    - inv.showMenu() displays all items
3. Item Selection Loop
    - User enters Item ID and quantity
    - Order::addItem() adds items to order
    - Loop continues until user enters 0
4. Bill Calculation
    - Order::calculateTotal() computes total
5. Payment Processing
    - User selects payment method
    - new CashPayment() or new UpiPayment() creates payment object
    - paymentMethod->makePayment() processes payment (polymorphism)
6. Inventory Update
    - If payment successful:
        - inv.updateItemStock() reduces quantities

        - Static members updated
        - Order confirmed
    - If payment fails:
        - Order canceled
        - Inventory unchanged.

Learning Outcomes

This project demonstrates:

1. Class Design: Proper encapsulation and data hiding
2. Inheritance Hierarchy: Base and derived classes with polymorphism
3. Template Programming: Generic programming with Inventory template
4. Operator Overloading: Custom operators for intuitive syntax
5. Memory Management: Dynamic allocation and deallocation
6. Static Members: Shared data across class instances
7. Friend Functions: Controlled access for specific operations
8. Error Handling: Robust input validation and exception handling
9. Software Architecture: Multi-file project organization
10. Real-world Application: Practical billing system implementation

---

Conclusion

This Cafeteria Billing and Inventory System showcases comprehensive implementation of Object-Oriented Programming principles in C++. The system provides a practical, functional solution for cafeteria management while demonstrating key OOP concepts including encapsulation, inheritance, polymorphism, template programming, operator overloading, and dynamic memory management

---

Course: Object-Oriented Programming
Language: C++
Concepts: Classes, Inheritance, Polymorphism, Templates, Operator Overloading, Dynamic Memory Management, Static Members

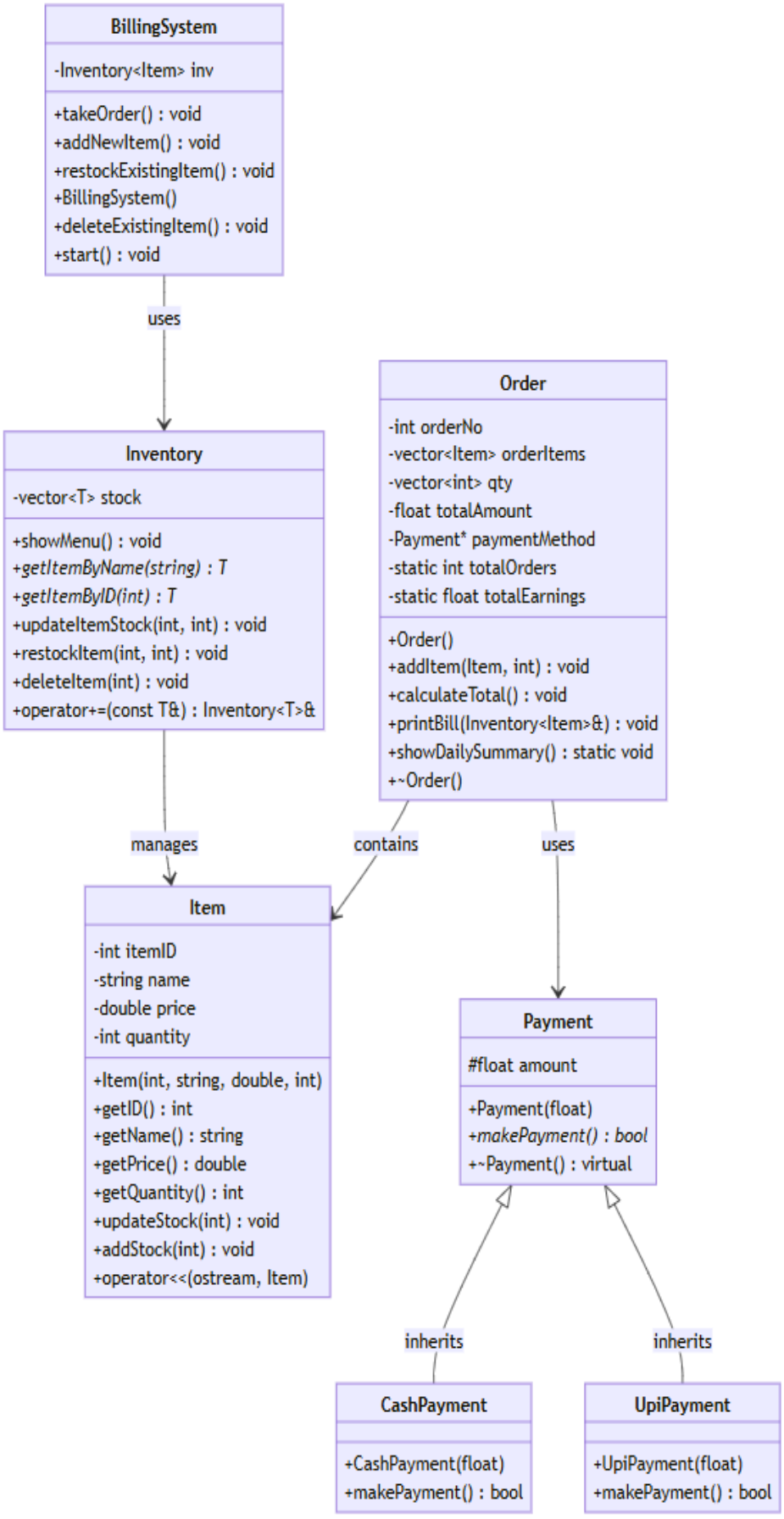
---

Contributions:

Implemented C++ code:

1. Qamar Abbas(CS24B1068):
    - item.h
    - item.cpp
    - payement.h
    - payment.cpp
2. Pruthvi Ram(CS24B1022):
    - order.cpp
    - order.h
    - inventory.cpp
    - inventory.h
3. V.Vinay(CS24B1040):
    - bill.h
    - bill.cpp
    - main.cpp

Report phase 1: V.Vinay(CS24B1040)
Report phase 2: V.Vinay(CS24B1040)
Powerpoint presentation: Pruthvi Ram(CS24B1022)


BillingSystem
-Inventory<Item> inv
+takeOrder() : void
+addNewItem() : void
+restockExistingItem() : void
+BillingSystem()
+deleteExistingItem() : void
+start() : void
uses
Inventory
-vector<T> stock
+showMenu() : void
+getItemByName(string) : T
+getItemByID(int) : T
+updateItemStock(int, int) : void
+restockItem(int, int) : void
+deleteItem(int) : void
+operator+=(const T&) : Inventory<T>&
Order
-int orderNo
-vector<Item> orderItems
-vector<int> qty
-float totalAmount
-Payment* paymentMethod
-static int totalOrders
-static float totalEarnings
+Order()
+addItem(Item, int) : void
+calculateTotal() : void
+printBill(Inventory<Item>&) : void
+showDailySummary() : static void
+~Order()
manages
contains
uses
Item
-int itemID
-string name
-double price
-int quantity
+Item(int, string, double, int)
+getID() : int
+getName() : string
+getPrice() : double
+getQuantity() : int
+updateStock(int) : void
+addStock(int) : void
+operator<<(ostream, Item)
Payment
#float amount
+Payment(float)
+makePayment() : bool
+~Payment() : virtual
inherits
inherits
CashPayment
+CashPayment(float)
+makePayment() : bool
UpiPayment
+UpiPayment(float)
+makePayment() : bool

Sample Outputs

```
        CAFETERIA BILLING SYSTEM
1. Show Menu
2. Take Order
3. Daily Summary
4. Add New Item
5. Restock Item
6. Delete Item
7.Exit
Choice: 1

        MENU
[101] Burger  Rs.80 (Available: 20)
[102] Pizza  Rs.150 (Available: 10)
[103] Fries  Rs.60 (Available: 15)
[104] Coke  Rs.40 (Available: 30)


        CAFETERIA BILLING SYSTEM
1. Show Menu
2. Take Order
3. Daily Summary
4. Add New Item
5. Restock Item
6. Delete Item
7.Exit
Choice: 2

        Take Order

        MENU
[101] Burger  Rs.80 (Available: 20)
[102] Pizza  Rs.150 (Available: 10)
[103] Fries  Rs.60 (Available: 15)
[104] Coke  Rs.40 (Available: 30)

Enter Item ID (0 to finish): 101
Enter quantity: 2
Selected: [101] Burger  Rs.80 (Available: 20)
```

```
Enter Item ID (0 to finish): 101
Enter quantity: 2
Selected: [101] Burger  Rs.80 (Available: 20)

Enter Item ID (0 to finish): 0

        BILL
Order No: 1
- Burger x2 = Rs.160
Total: Rs.160

Payment: 1. Cash  2. UPI
Choice: 1
Enter cash given: 150
Insufficient amount. Need at least Rs.160
Enter cash given: 200
Change Returned: Rs.40

Payment Successful. Order Confirmed.


        CAFETERIA BILLING SYSTEM
1. Show Menu
2. Take Order
3. Daily Summary
4. Add New Item
5. Restock Item
6. Delete Item
7.Exit
Choice: 2

        Take Order

        MENU
[101] Burger  Rs.80 (Available: 18)
[102] Pizza  Rs.150 (Available: 10)
[103] Fries  Rs.60 (Available: 15)
[104] Coke  Rs.40 (Available: 30)
```

```
[103] Fries  Rs.60 (Available: 15)
[104] Coke  Rs.40 (Available: 30)

Enter Item ID (0 to finish): 102
Enter quantity: 2
Selected: [102] Pizza  Rs.150 (Available: 10)

Enter Item ID (0 to finish): 0

        BILL
Order No: 2
- Pizza x2 = Rs.300
Total: Rs.300

Payment: 1. Cash  2. UPI
Choice: 2
Enter UPI ID : 102563
Processing UPI payment Success

Payment Successful. Order Confirmed.


        CAFETERIA BILLING SYSTEM
1. Show Menu
2. Take Order
3. Daily Summary
4. Add New Item
5. Restock Item
6. Delete Item
7.Exit
Choice: 3

        DAILY SUMMARY
------------------------
Total Orders: 2
Total Earnings: Rs.460


        CAFETERIA BILLING SYSTEM
```

```
        CAFETERIA BILLING SYSTEM
1. Show Menu
2. Take Order
3. Daily Summary
4. Add New Item
5. Restock Item
6. Delete Item
7.Exit
Choice: 4

        Add New Item
Enter new Item ID: 101
Error: Item ID 101 already exists.


        CAFETERIA BILLING SYSTEM
1. Show Menu
2. Take Order
3. Daily Summary
4. Add New Item
5. Restock Item
6. Delete Item
7.Exit
Choice: 4

        Add New Item
Enter new Item ID: 105
Enter name: juice
Enter price: 50
Enter initial stock: 10
Item added successfully.


        CAFETERIA BILLING SYSTEM
1. Show Menu
2. Take Order
3. Daily Summary
4. Add New Item
5. Restock Item
```

```
        CAFETERIA BILLING SYSTEM
1. Show Menu
2. Take Order
3. Daily Summary
4. Add New Item
5. Restock Item
6. Delete Item
7.Exit
Choice: 1

        MENU
[101] Burger  Rs.80 (Available: 18)
[102] Pizza  Rs.150 (Available: 8)
[103] Fries  Rs.60 (Available: 15)
[104] Coke  Rs.40 (Available: 30)
[105] juice  Rs.50 (Available: 10)


        CAFETERIA BILLING SYSTEM
1. Show Menu
2. Take Order
3. Daily Summary
4. Add New Item
5. Restock Item
6. Delete Item
7.Exit
Choice: 5

        Restock Item

        MENU
[101] Burger  Rs.80 (Available: 18)
[102] Pizza  Rs.150 (Available: 8)
[103] Fries  Rs.60 (Available: 15)
[104] Coke  Rs.40 (Available: 30)
[105] juice  Rs.50 (Available: 10)
```

```
[104] Coke  Rs.40 (Available: 30)
[105] juice  Rs.50 (Available: 10)

Enter Item ID to restock: 105
Enter quantity to add: 20
Stock added successfully.
Stock updated.


        CAFETERIA BILLING SYSTEM
1. Show Menu
2. Take Order
3. Daily Summary
4. Add New Item
5. Restock Item
6. Delete Item
7.Exit
Choice: 5

        Restock Item

        MENU
[101] Burger  Rs.80 (Available: 18)
[102] Pizza  Rs.150 (Available: 8)
[103] Fries  Rs.60 (Available: 15)
[104] Coke  Rs.40 (Available: 30)
[105] juice  Rs.50 (Available: 30)

Enter Item ID to restock: 108
Error: Item ID not found.


        CAFETERIA BILLING SYSTEM
1. Show Menu
2. Take Order
3. Daily Summary
4. Add New Item
5. Restock Item
6. Delete Item
```

```
6. Delete Item
7.Exit
Choice: 6

        Delete Item

        MENU
[101] Burger  Rs.80 (Available: 18)
[102] Pizza  Rs.150 (Available: 8)
[103] Fries  Rs.60 (Available: 15)
[104] Coke  Rs.40 (Available: 30)
[105] juice  Rs.50 (Available: 30)

Enter Item ID to delete: 104
Item deleted successfully.


        CAFETERIA BILLING SYSTEM
1. Show Menu
2. Take Order
3. Daily Summary
4. Add New Item
5. Restock Item
6. Delete Item
7.Exit
Choice: 1

        MENU
[101] Burger  Rs.80 (Available: 18)
[102] Pizza  Rs.150 (Available: 8)
[103] Fries  Rs.60 (Available: 15)
[105] juice  Rs.50 (Available: 30)


        CAFETERIA BILLING SYSTEM
1. Show Menu
2. Take Order
3. Daily Summary
4. Add New Item
```